EJERCICIO 1

Escribe un subprograma en Pascal que calcule el número de nodos de un árbol binario. Analiza la complejidad de tu solución y determina si es un subprograma que debe añadirse al TAD o por el contrario debería ser externo al mismo.

SOLUCIÓN:

```
function node_count(a:tBinarySearchTree): integer;
begin
    if (a = NIL) then node_count := 0
    else node_count := 1 + node_count(a^.hd) + node_count(a^.hi)
end;
```

El subprograma tiene complejidad O(n) dado que visita todos los nodos una vez, y debe añadirse al TAD puesto que debe conocer su representación interna.

Comentario: En un árbol vacío no hay ningún nodo: esa es la condición de salida o caso base de la recursividad. En cualquier otro árbol, el número de nodos total es 1 (el nodo raíz) + el número de nodos del subárbol izquierdo + el número de nodos del subárbol derecho. El cómputo del número de nodos de estos dos subárboles se realiza igual que el cómputo sobre el árbol completo y por tanto son llamadas a la misma función (`node_count`) pero en lugar de sobre el árbol completo sobre los subárboles hijo izquierdo e hijo derecho.

EJERCICIO 2

Añade al TAD BinarySearchTree recorridos limitados a n elementos. Un ejemplo de recorrido limitado sería un recorrido en inorden que solo visita los 10 primeros nodos del árbol. Obviamente, si el árbol no tiene tantos elementos como marca el límite, tu procedimiento visitaría únicamente tantos nodos como contenga el árbol.

SOLUCIÓN:

```
procedure limited_preorder(a: tBinarySearchTree; var limit: integer);
begin
  if (a <> NIL) and (limit>0) then begin
    visit(a^.info);
    limit:= limit-1;
    limited_preorder(a^.hi,limit); { II }
    limited_preorder(a^.hd,limit)  { II }
  end
end;

procedure limited_inorder(a: tBinarySearchTree; var limit: integer);
begin
  if (a <> NIL) and (limit>0) then begin
    limited_inorder(a^.hi, limit); { I }
    if limit>0 then
    begin
      visit(a^.info);
      limit:= limit-1
    end;
    limited_inorder(a^.hd,limit) { II }
  end
end;

procedure limited_postorder(a: tBinarySearchTree; var limit: integer);
begin
```

```
  if (a <> NIL) and (limit>0) then begin
    limited_postorder(a^.hi, limit); { I }
    limited_postorder(a^.hd, limit); { I }
    if limit>0 then
      begin
        visit(a^.info);
        limit:= limit-1
      end
  end
end;
```

Comentario: En los 3 enfoques utilizamos una variable límite que pasamos por referencia. Esta variable marcará si aún quedan valores por mostrar o ya hemos cumplido con el valor establecido inicialmente. La variable se decrementará cada vez que se hace una visita efectiva de un nodo y se llamará recursivamente a la función con el valor modificado (los dos casos de preorder y el segundo de inorden, marcados en comentario como (I)) o con el mismo que se recibe (caso del primer caso de inorden y los dos de postorden, marcados en comentario como (II)) con lo cual la función que recibe la llamada recursiva tendrá el mismo valor (caso II) o un valor – 1 (caso II) que la función la que invoca.

EJERCICIO 3

Añadir al TAD BinarySearchTree una función que retorne el valor del nodo máximo del árbol. Analizar su complejidad y compararla con una función equivalente en un árbol binario general (no de búsqueda).

SOLUCIÓN:

```
[En un BST]

function max_node(a:tBinarySearchTree): integer;
begin
  if a=NIL then max_node:=-MAXINT
  else if a^.hd<> NIL then max_node:= max_node(a^.hd)
  else max_node:= a^.info
end;

[En un árbol general]

function max_node(a:tBinaryTree): integer;
begin
  if a=NIL then max_node:=-MAXINT
  else max_node:= max(a^.info, max(max_node(a^.hi),max_node(a^.hd)))
end;
```

En el caso del árbol binario la complejidad es O(n) dado que hay que visitar todos los nodos. En un árbol binario de búsqueda, solo se visita la rama más a la derecha y por tanto la complejidad es mucho menor si bien nunca lineal: O(log(n)).

Comentario: El máximo de un árbol vacío es el menor valor posible (-MAXINT). En otro árbol, el máximo deberá ser calculado bien como el máximo entre el valor en la raíz y los máximos valores encontrados en el subárbol del hijo izquierdo y en el subárbol hijo derecho (árbol binario general) o bien como el valor más a la derecha abajo del árbol (árbol binario de búsqueda BST). Para la implementación en un árbol binario general nos apoyamos en la función `max` de la biblioteca `Math` pero de no conocer dicha función se puede programar una sencilla función que haga lo mismo.

EJERCICIO 4

Se dice que un árbol binario está lleno cuando todos sus nodos tienen 2 hijos o ninguno (nodos hoja). Escribe un subprograma en Pascal que determine si un árbol está lleno y calcula la complejidad de tu solución.

SOLUCIÓN:

```
function is_full(a:tBinarySearchTree): boolean;
begin
    if (a = NIL) or ((a^.hd = NIL) and (a^.hi = NIL))then is_full:= true
    else if (a^.hd <> NIL) and (a^.hi <> NIL) then is_full:= is_full(a^.hd) and  is_full(a^.hi)
    else is_full:= false
end;
```

Una vez realizado el ejercicio, te proponemos un análisis adicional: Hemos encontrado por Internet la siguiente implementación de la función que comprueba si un árbol está lleno.

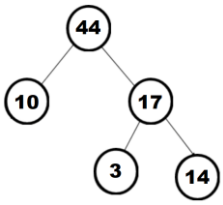
```
function arbol_lleno(a:tBinarySearchTree): boolean;
begin
    if a = NIL then arbol_lleno:= true
    else if altura(a^.hi) <> altura(a^.hd) then arbol_lleno:= false
    else arbol_lleno:= arbol_lleno(a^.hi) and arbol_lleno(a^.hd)
end;
```

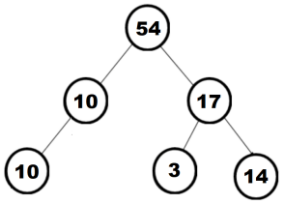
Presumimos que no es óptima y por tanto necesitamos analizar su complejidad en notación O grande. Estudia su complejidad y comenta los resultados.

Cada llamada recursiva a `arbol_lleno()` implica llamar a altura() desde el siguiente nivel del árbol. Puesto que la función altura() recorre todos los nodos desde ese nivel del árbol hacia abajo, la complejidad será por tanto O(n log(n)) para un árbol lleno de n nodos.

EJERCICIO 5

Escribe un método que verifique si un árbol binario es un árbol "suma". Los árboles "suma" tienen la característica de que todos sus nodos no hoja son iguales a la suma de sus nodos descendientes. Por ejemplo, los siguientes son árboles "suma":





SOLUCIÓN:

```
function suma_nodos(a:tBinaryTree): integer;
begin
  if is_empty(a) then suma_nodos := 0
  else suma_nodos:= suma_nodos(a^.hi) + a^.info + suma_nodos(a^.hd)
end;

function es_hoja(a:tBinaryTree): boolean;
begin
  es_hoja:= (a<>NIL) and (a^.hd = a^.hi) // si ambos son iguales solo pueden ser = NIL
end;
```

```
function es_arbol_suma(a:tBinarySearchTree): boolean;
var
  suma_dch, suma_izq: integer;
begin
  if is_empty(a) or es_hoja(a) then es_arbol_suma:= true
  else
    begin
      suma_izq := suma_nodos(a^.hi);
      suma_dch := suma_nodos(a^.hd);
      es_arbol_suma := (a^.info = suma_izq + suma_dch) and es_arbol_suma(a^.hi) and es_arbol_suma(a^.hd)
    end
end;
```

EJERCICIO 6

Dado un árbol binario con elementos enteros, elabora un subprograma que ponga como valor de la raíz la suma de los valores de todos sus descendientes. Tu solución debe tener una *complejidad de tiempo máxima: O(n)* y emplear un *espacio auxiliar: O(1).*

SOLUCIÓN:

```
procedure pon_en_raiz_suma(var a: tBinaryTree);
begin
  if (a<>NIL) then a^.info:= suma_nodos(a^.hi) + suma_nodos(a^.hd)
end;

function suma_nodos(a:tBinaryTree): integer;
begin
  if a=NIL then suma_nodos:=0
  else suma_nodos:= a^.info + suma_nodos(a^.hi) + suma_nodos(a^.hd)
end;
```

EJERCICIO 7

Una gasolinera almacena pagos asociados a la matrícula del vehículo que reposta de cara a su programa de puntos. Los pagos están almacenados en una lista enlazada cuyo orden es el orden en que se produjeron los pagos. Cada pago está formado por información sobre el tipo de combustible (`diesel`, `diesel_plus`, `gasolina95`, `gasolina98`), importe y matrícula del vehículo. Crea un árbol resumen con los totales por matrícula en el que se pueda buscar por matrícula.

SOLUCIÓN:

```
program ejercicio7;

uses uListaPagos, uBSTGenerico, uPago, uTElemento;

procedure procesar_pagos(var lista_pagos: tlistaPagos; var arbol_resumen: tBSTGenerico);
var
  pago: tPago;
  nuevo_registro: tElemento;
  registro_en_arbol: ^tElemento;
begin
  while not is_empty(lista_pagos) do
  begin
    pago:= first(lista_pagos);
    nuevo_registro.total:= pago.importe;
    nuevo_registro.matricula:= pago.matricula;
    registro_en_arbol := get_clave(arbol_resumen, nuevo_registro);
    if registro_en_arbol = NIL then
       add(arbol_resumen, nuevo_registro)
```

```
    else
       registro_en_arbol^.total := registro_en_arbol^.total + pago.importe;
  end
end;

var
  lista_pagos: tlistaPagos;
  arbol_resumen: tBSTGenerico;

begin
  initialize(lista_pagos);
  // introducción de pagos en la gasolinera...
  procesar_pagos(lista_pagos, arbol_resumen);
  // resto del programa, si lo hay...
end.
```

Esta solución utiliza varias unidades cuyas interfaces son:

```
unit uListaPagos;

interface
  uses uPago;

  type

    tnodo = record
         info: tPago;
         sig: ^tnodo;
    end;

    tlistaPagos = record
      first, last: ^tnodo
    end;

  { Operaciones }
  procedure initialize(var list: tlistaPagos);
  function is_empty(list: tlistaPagos): boolean;
  function first(list: tlistaPagos): tPago;
  function last(list: tlistaPagos): tPago;
  procedure insert_at_end(var list: tlistaPagos; p:tPago);
  procedure insert_at_begin(var list: tlistaPagos; p:tPago);
  procedure delete_at_begin(var list: tlistaPagos);

unit uPago;

interface
   type

     tcombustible = (diesel, gas95, gas98, diesel_plus);

     tPago =record
       combustible: tcombustible;
       importe: real;
       matricula: string[7];
     end;

   procedure assign(var e1: tPago; e2: tPago);
   function less_than(e1: tPago; e2: tPago): boolean;
   function equal_to(e1: tPago; e2: tPago): boolean;
   procedure imprimir(e: tPago);

unit uBSTGenerico;

interface
  uses uTElemento;

  type
    ptElemento= ^tElemento;
```

```
    tBSTGenerico = ^tnode;
    tnode = RECORD
      info : tElemento;
      hi,hd : tBSTGenerico;
    end;

  procedure initializeBST(var a:tBSTGenerico);
  function is_emptyBST(a:tBSTGenerico): boolean;
  procedure add(var a:tBSTGenerico; clave: TElemento);
  function in_tree(a: tBSTGenerico; clave: TElemento): boolean;
  procedure remove(var a:tBSTGenerico; x: TElemento);
  function get_clave(a: tBSTGenerico; clave: TElemento): ptElemento;
```

```
unit uTElemento;

interface
   type

     tElemento=record
       total: real;
       matricula: string[7];
     end;

   procedure assign(var e1: tElemento; e2: TElemento);
   function less_than(e1: tElemento; e2: TElemento): boolean;
   function equal_to(e1: tElemento; e2: TElemento): boolean;
   procedure imprimir(e: tElemento);

implementation

   function less_than(e1: tElemento; e2: TElemento): boolean;
   begin
     less_than := e1.matricula < e2.matricula;
   end;

   // resto de implementaciones, trivial
```

La única novedad en todas estas unidades es la implementación de la función `get_clave()` en el árbol binario de búsqueda genérico, la cual busca una matrícula en el árbol y retorna un puntero (`ptElemento`) al elemento en el nodo del árbol si la encuentra o `NIL` si la matrícula no se encuentra en el árbol. Su implementación se muestra a continuación:

```
function get_clave(a: tBSTGenerico; clave: TElemento): ptElemento;
begin
  if a = NIL then get_clave := NIL
  else if less_than(a^.info, clave) then get_clave := get_clave(a^.hd, clave)
  else if less_than(clave, a^.info) then get_clave := get_clave(a^.hi, clave)
  else get_clave := @a^.info
end;
```

EJERCICIO 8

Genera 2.000 números reales aleatorios entre 0 y 99, utilizando una precisión de dos cifras decimales y almacénalos en un árbol binario de búsqueda que contenga un representante entero y asociada a dicha clave los números reales que se han generado con esa parte entera junto con el número de veces que aparece. Así por ejemplo si el primer número generado es 7.90768 lo primero es truncarlo a dos cifras decimales: 7.90. Después, se inserta en el árbol en la clave 7, de forma que ese 7 será un nodo principal en el árbol del que saldrá una lista de tuplas [valor,frecuencia] que inicialmente tendrá solo la tupla valor=7.90 y frecuencia=1. Si más tarde se genera el 7.353567 habrá que añadir 7.35 a la lista del nodo 7 en el árbol con

frecuencia 1. Si más tarde se genera el valor 7.3509812 habrá que ir a la lista que sale del nodo principal del árbol 7 e incrementar en 1 la frecuencia del 7.35, etc. Al final del proceso, se pide recorrer el árbol y mostrar el número entero que representa a la parte entera de los números generados de la cual ha habido más ocurrencias.

SOLUCIÓN:

```
program ejercicio_propuesto_arboles8;

uses uBSTListaReales;

const
  MAX_NUMEROS = 2000;

var
  a: tBSTListaReales;
  i, representante_entero: integer;
  real_x_100: real; // Pongo el real mult. por 100 para eludir problemas de redondeo

function genera_aleatorio: real;
begin
    genera_aleatorio:= int(random*1000);
end;

begin
  randomize;
  initialize(a);
  for i:=1 to MAX_NUMEROS do begin
       real_x_100:= genera_aleatorio;
       representante_entero:= trunc(real_x_100/100);
       add(a,representante_entero,real_x_100);
      // writeln(real_x_100)
  end;
  writeln('El entero mas frecuente es: ', mas_frecuente(a).representante);
  inorder(a);
  readln; // Muestro el árbol para verificar resultados
end.
```

Esta solución utiliza las siguientes unidades:

```
unit uBSTListaReales;

interface

  uses
    uListaDuplas;

  type

    tBSTListaReales = ^tnode;

    tnode = RECORD
      info : integer;
      hi, hd : tBSTListaReales;
      lista_reales: tlistaDuplas;
    end;

    par_frecuencias=record
      representante, frecuencia: integer;
    end;

  procedure initialize(var a:tBSTListaReales);
  function is_empty(a:tBSTListaReales): boolean;
  procedure add(var a:tBSTListaReales; representante: integer; valor_real: real);
  function in_tree(a: tBSTListaReales; clave: integer): boolean;
  procedure inorder(a: tBSTListaReales);
  function mas_frecuente(a:tBSTListaReales): par_frecuencias;
```

```
implementation

uses
  Math;

procedure initialize(var a:tBSTListaReales);
begin
  a:= NIL;
end;

function is_empty(a:tBSTListaReales): boolean;
begin
  is_empty:= a=NIL;
end;

function in_tree(a: tBSTListaReales; clave: integer): boolean;
begin
  if a = NIL then in_tree := FALSE
  else if a^.info < clave then in_tree := in_tree(a^.hd, clave)
  else if a^.info > clave then in_tree := in_tree(a^.hi, clave)
  else in_tree := TRUE
end;

procedure add(var a:tBSTListaReales; representante: integer; valor_real: real);
begin
  if a = NIL then begin
    new(a);
    a^.info := representante;
    uListaDuplas.initialize(a^.lista_reales);
    insert(a^.lista_reales,valor_real);
    a^.hi := NIL;
    a^.hd := NIL;
  end
  else if a^.info < representante then add(a^.hd, representante, valor_real)
  else if a^.info > representante then add(a^.hi, representante, valor_real)
  else insert(a^.lista_reales,valor_real);
end;

procedure inorder(a: tBSTListaReales);
begin
  if (a <> NIL) then begin
    inorder(a^.hi);
    writeln('----- ', a^.info, ' -----');
    show(a^.lista_reales);
    inorder(a^.hd)
  end
end;

function mas_frecuente(a:tBSTListaReales): par_frecuencias;
var
  izquierdo, derecho, raiz, mayor: par_frecuencias;
begin
  mas_frecuente.frecuencia:=0;
  mas_frecuente.representante:=0;
  if a <> NIL then begin
    izquierdo:= mas_frecuente(a^.hi);
    derecho:= mas_frecuente(a^.hd);
    raiz.representante:= a^.info;
    raiz.frecuencia:= max_frecuencia(a^.lista_reales);
    mayor := raiz;
    if mayor.frecuencia < izquierdo.frecuencia then mayor:= izquierdo;
    if mayor.frecuencia  <  derecho.frecuencia then mayor:= derecho;
    mas_frecuente:= mayor;
  end;
end;
```

```
unit uListaDuplas;

interface
  type

    tdupla=record
      valor: real;
      frecuencia: integer;
    end;

    tnodo = record
      dupla: tdupla;
      next: ^tnodo;
    end;

    tlistaDuplas = record
      first, last: ^tnodo
    end;

  procedure initialize(var list: tlistaDuplas);
  function is_empty(list: tlistaDuplas): boolean;
  function in_list(list: tlistaDuplas; x:real): boolean;
  procedure insert(var list: tlistaDuplas; x: real);
  procedure show(list: tlistaDuplas);
  function max_frecuencia(list: tlistaDuplas): integer;

implementation

  procedure initialize(var list: tlistaDuplas);
  begin
    list.first := nil;
    list.last := nil;
  end;

  function is_empty(list: tlistaDuplas): boolean;
  begin
    is_empty:= (list.first = nil) and (list.last = nil);
  end;

  function in_list(list: tlistaDuplas; x:real): boolean;
  var
    encontrado: boolean;
    aux:^tnodo;
  begin
    encontrado:= false;
    if not is_empty(list) then begin
        aux:= list.first;
        while (not encontrado) and (aux<>nil) do begin
          encontrado:= x = aux^.dupla.valor;
          aux:= aux^.next;
        end;
      end;
    in_list:= encontrado;
  end;

  procedure insert(var list: tlistaDuplas; x: real);
  var
    new_node, actual: ^tnodo;
  begin
    new(new_node);
    new_node^.dupla.valor:= x;
    new_node^.dupla.frecuencia:= 1;
    if is_empty(list) or (x < list.first^.dupla.valor) then begin // insertar el primero
        new_node^.next:= list.first;
        list.first:= new_node;
      end
    else begin  // Situar actual en el nodo anterior a la inserción
        actual:= list.first;
```

```
        while (actual^.next <> nil) and (x > actual^.next^.dupla.valor) do
            actual:= actual^.next;
        // En este punto el valor más a la derecha o es nil, o es igual a x o es mayor que x
        if (actual^.next = nil) or (actual^.next^.dupla.valor > x) then begin
          new_node^.next:= actual^.next;
          actual^.next:= new_node;
        end
        else begin // no hay que insertar nuevo nodo sino modificar uno existente
          actual^.next^.dupla.frecuencia:= actual^.next^.dupla.frecuencia + 1;
          dispose(new_node);
        end;
    end;
  end;

  function max_frecuencia(list: tlistaDuplas): integer;
  var
    actual: ^tnodo;
  begin
    actual:= list.first;
    max_frecuencia:= -MAXINT;
    while (actual <> nil) do begin
        if actual^.dupla.frecuencia > max_frecuencia then
           max_frecuencia:= actual^.dupla.frecuencia;
        actual:= actual^.next;
    end;
  end;

  procedure show(list: tlistaDuplas);
  var
    aux: ^tnodo;
  begin
    if is_empty(list) then writeln('Empty list: No elements to show')
    else begin
      aux:= list.first;
      while aux<>nil do begin
        writeln(aux^.dupla.valor,' - Frecuencia: ',aux^.dupla.frecuencia);
        aux:= aux^.next
      end
    end
  end;

end.
```

EJERCICIO 9 (reto)

Escribe un programa que lea un texto de longitud indeterminada y que produzca como resultado un listado por pantalla de todas las palabras diferentes contenidas en el texto en orden alfabético junto con su frecuencia de aparición.

El texto original se obtiene de un fichero de texto, para lo cual es recomendable descargar cualquier libro electrónico de la Biblioteca Virtual Miguel de Cervantes (https://www.cervantesvirtual.com/), los cuales suelen tener cientos de miles de palabras.

Emplea para tu solución un árbol binario de búsqueda, donde cada nodo contenga una palabra y su frecuencia y compara la eficiencia de esta solución con la de una implementación que utilizase una lista ordenada para almacenar la información. Reflexiona al respecto.

[No se proporciona solución a este problema]